

LAB ASSIGNMENT-19.3

2303A54012
A.Sanjay B47A

Lab 19 – Code Translation: Converting Between Programming Languages

Task 1 – Python to C++ Conversion

Task:

Translate a simple Python class into C++ using AI assistance.

Instructions:

- Prompt AI to convert a Python class representing a Student with attributes and methods into equivalent C++ code
- Ensure proper handling of:
 - Constructors
 - Data types
 - Access specifiers
- Compile and run both versions to verify output consistency

Expected Output:

An equivalent working C++ class translated from Python.

Python Implementation:

First, define the source Python class. This class uses a constructor (`__init__`) and a simple method to display data.

C++ Implementation:

The AI-generated C++ code handles memory and type safety explicitly, which is a key difference between the two languages.

Prompt:

"Convert the following Python Student class into C++. Ensure you use a proper constructor, map data types correctly (string, int, double), and use access specifiers (private for attributes, public for methods). Include a main function to demonstrate the output."

Code in python:

```

1 # Python Source Code
2 class Student:
3     def __init__(self, name, roll_number, gpa):
4         self.name = name # String attribute
5         self.roll_number = roll_number # Integer attribute
6         self.gpa = gpa # Float attribute
7
8     def display_details(self):
9         print(f"Student Name: {self.name}")
10        print(f"Roll Number: {self.roll_number}")
11        print(f"GPA: {self.gpa}")
12
13 # Testing the class
14 s1 = Student("Aryan Sharma", 101, 8.5)
15 s1.display_details()

```

Output:

```

Student Name: Aryan Sharma
Roll Number: 101
GPA: 8.5

...Program finished with exit code 0
Press ENTER to exit console.

```

Code in C++:

```

1 #include <iostream>
2 #include <string>
3 class Student {
4 private:
5     // Attributes (Access Specifier: private)
6     std::string name;
7     int roll_number;
8     double gpa;
9 public:
10    // Constructor (Proper handling of initialization)
11    Student(std::string n, int r, double g) {
12        name = n;
13        roll_number = r;
14        gpa = g;
15    }
16    // Method to display details (Access Specifier: public)
17    void display_details() {
18        std::cout << "Student Name: " << name << std::endl;
19        std::cout << "Roll Number: " << roll_number << std::endl;
20        std::cout << "GPA: " << gpa << std::endl;
21    }
22 };
23 int main() {
24    // Instantiating the class
25    Student s1("Aryan Sharma", 101, 8.5);
26    s1.display_details();
27    return 0;
28 }

```

Output:

```

Student Name: Aryan Sharma
Roll Number: 101
GPA: 8.5

...Program finished with exit code 0
Press ENTER to exit console.

```

Explanation & Comparison:

In your report, include these observations regarding the translation:

- Access Specifiers: Python does not have strict access control (everything is public by default), whereas C++ requires explicit private and public blocks to enforce encapsulation.
- Data Types: Python is dynamically typed; C++ is statically typed. We had to explicitly define `std::string`, `int`, and `double`.
- Constructors: In Python, the constructor is `__init__`. In C++, the constructor has the same name as the class and no return type.
- Syntax: Python uses indentation to define scope, while C++ uses curly braces `{}` and semicolons `;`.

Task 2 – Java to Python Function Conversion

Task:

Convert a Java method that checks whether a number is prime into Python.

Instructions:

- Ask AI to translate a Java isPrime() method into Python
- Test the function with multiple inputs
- Ensure the Python version follows Pythonic syntax and logic

Expected Output:

A correct Python function equivalent to the original Java prime-checking method.

Java Implementation

This is the standard Java method using a for loop and the Math.sqrt() function for efficiency.

Python Implementation

The AI simplifies the syntax by removing curly braces and using the range object.

Prompt:

"Translate this Java isPrime method into Python. Ensure the resulting code is 'Pythonic'—use the math module for the square root, use a range-based loop, and include a list of test cases to verify the output for multiple numbers."

Code :

```
1 // Java Source Code
2 public class PrimeTask {
3     public static boolean isPrime(int n) {
4         if (n <= 1) return false;
5         for (int i = 2; i <= Math.sqrt(n); i++) {
6             if (n % i == 0) return false;
7         }
8         return true;
9     }
10
11     public static void main(String[] args) {
12         System.out.println("7 is prime: " + isPrime(7));
13     }
14 }
```

```

1 import math
2
3 def is_prime(n):
4     """
5     Checks if a number is prime.
6     Following Pythonic logic: handles edge cases and uses an optimized loop.
7     """
8     if n <= 1:
9         return False
10
11     # We check up to the square root of n for efficiency
12     for i in range(2, int(math.sqrt(n)) + 1):
13         if n % i == 0:
14             return False
15     return True
16
17 # --- Testing with multiple inputs ---
18 test_numbers = [2, 4, 7, 15, 19, 25, 29]
19 print("Number | Is Prime?")
20 print("-" * 18)
21 for num in test_numbers:
22     print(f"{num:6} | {is_prime(num)}")

```

Output:

```

Number | Is Prime?
-----
    2 | True
    4 | False
    7 | True
   15 | False
   19 | True
   25 | False
   29 | True

...Program finished with exit code 0
Press ENTER to exit console.

```

Explanation & Comparison:

In your report, highlight these key differences found during the translation:

- **Type Declaration:** Java requires `int n` and `boolean` return types. Python is dynamically typed, so no types are declared.
- **Loop Syntax:** Java uses `for (int i = 2; i <= limit; i++)`. Python uses `for i in range(2, limit + 1)`, which is more readable but requires adding `+1` because the stop value is exclusive.
- **Booleans:** Java uses lowercase `true/false`, while Python uses capitalized `True/False`.
- **Library Import:** Python requires `import math` to use `math.sqrt()`, whereas Java includes it in the standard `java.lang` package.

Task 3 – Pseudocode to Python Implementation

Task:

Translate a given pseudocode algorithm into Python using AI.

Instructions:

- Provide AI with pseudocode for sorting numbers (e.g., bubble sort or selection sort)
- Ask AI to convert it into executable Python code
- Validate correctness using sample input lists

Expected Output:

A working Python program that correctly implements the pseudocode logic.

Pseudocode (Input)

Pseudocode is a simplified, language-independent way to write algorithms. You will provide this to the AI:

Python Implementation

The AI translates the abstract "FOR" loops and "SWAP" commands into valid Python syntax.

Prompt:

"I am providing pseudocode for a Selection Sort algorithm. Please translate this into executable Python code. Ensure you use descriptive variable names, include comments explaining the logic, and provide a sample list of numbers to test and validate the sorting results."

Code:

```
1  PROCEDURE selectionSort(list)
2      n = length(list)
3      FOR i FROM 0 TO n-1
4          min_index = i
5          FOR j FROM i+1 TO n-1
6              IF list[j] < list[min_index] THEN
7                  min_index = j
8              END IF
9          END FOR
10         SWAP list[i] with list[min_index]
11     END FOR
12 END PROCEDURE
```

```

1 def selection_sort(arr):
2     """
3     Implementation of Selection Sort translated from pseudocode.
4     """
5     n = len(arr)
6
7     # Outer Loop to move the boundary of the unsorted subarray
8     for i in range(n):
9         # Find the minimum element in the remaining unsorted array
10        min_index = i
11        for j in range(i + 1, n):
12            if arr[j] < arr[min_index]:
13                min_index = j
14
15        # Swap the found minimum element with the first element
16        # Pythonic swap: no temporary variable needed
17        arr[i], arr[min_index] = arr[min_index], arr[i]
18
19    return arr
20
21 # --- Validation with Sample Input ---
22 sample_data = [64, 25, 12, 22, 11]
23 print("Original List:", sample_data)
24
25 sorted_data = selection_sort(sample_data.copy())
26 print("Sorted List: ", sorted_data)
27
28 # Verification check
29 if sorted_data == sorted(sample_data):
30     print("\nResult: SUCCESS! The algorithm correctly sorted the list.")
31 else:
32     print("\nResult: FAILURE. The sorting logic is incorrect.")

```

Output:

```

Original List: [64, 25, 12, 22, 11]
Sorted List:  [11, 12, 22, 25, 64]

Result: SUCCESS! The algorithm correctly sorted the list.

...Program finished with exit code 0
Press ENTER to exit console.

```

Explanation & Logic Mapping

In your report, map the pseudocode commands to the Python code:

- length(list) -> rightarrow -> became len(arr).
- FOR i FROM 0 TO n-1 -> rightarrow -> became for i in range(n).
- SWAP list[i] with list[min_index] -> rightarrow -> Python handles this elegantly with tuple unpacking: a, b = b, a.
- END FOR / END IF -> rightarrow -> In Python, these are replaced by indentation levels, making the code cleaner.

Task 4 – SQL to Pandas Query

Task:

Translate a SQL query into a Pandas DataFrame operation in Python.

Instructions:

- Provide AI with a SQL query (e.g., SELECT name, salary FROM employees WHERE salary > 50000).
- Ask AI to generate equivalent Pandas code.
- Test the generated code on a sample DataFrame.

Expected Output:

- Equivalent Pandas query that retrieves the correct subset of data.

Prompt:

"Translate the following SQL query into Python Pandas code: 'SELECT name, salary FROM employees WHERE salary > 50000'. First, create a sample DataFrame called 'employees' with columns 'id', 'name', 'salary', and 'department'. Then, show the equivalent Pandas operation to filter and select the specific columns. Explain the mapping between the SQL keywords and Pandas methods."

SQL Query (Input)

We will use a standard filtering and column selection query:

```
1 SELECT name, salary
2 FROM employees
3 WHERE salary > 50000;
```

Pandas Implementation

The AI-generated code uses boolean indexing, which is the most common way to filter data in Pandas.

```
1 import pandas as pd
2
3 # Step 1: Create a sample DataFrame
4 data = {
5     'id': [1, 2, 3, 4, 5],
6     'name': ['Amit', 'Priya', 'Rahul', 'Sonia', 'Vikram'],
7     'salary': [45000, 60000, 52000, 30000, 75000],
8     'department': ['HR', 'IT', 'IT', 'Sales', 'Engineering']
9 }
10
11 employees = pd.DataFrame(data)
12
13 # Step 2: Equivalent Pandas Query
14 # SQL: SELECT name, salary FROM employees WHERE salary > 50000
15 result = employees[employees['salary'] > 50000][['name', 'salary']]
16
17 # Step 3: Display results
18 print("Original DataFrame:")
19 print(employees)
20 print("\nFiltered Result (Salary > 50000):")
21 print(result)
```

Output:

```
Original DataFrame:
   id  name  salary  department
0   1  Amit   45000          HR
1   2  Priya  60000          IT
2   3  Rahul  52000          IT
3   4  Sonia  30000        Sales
4   5  Vikram 75000  Engineering

Filtered Result (Salary > 50000):
   name  salary
1  Priya  60000
2  Rahul  52000
4  Vikram 75000

...Program finished with exit code 0
Press ENTER to exit console.
```

SQL to Pandas Mapping Logic

For your report, use this table to document how the translation works:

SQL Keyword	Pandas Equivalent	Explanation
FROM employees	employees	Refers to the DataFrame object name.
WHERE salary > 50000	employees['salary'] > 50000	Boolean indexing to filter rows.
SELECT name, salary	[['name', 'salary']]	Subsetting the DataFrame by column names.

Explanation:

SQL to Pandas (Data Manipulation)

- **Declarative vs. Imperative:** SQL is **declarative** (you tell the computer *what* you want). Pandas is **imperative/procedural** (you tell the computer *how* to get it).
- **Filtering:** The WHERE clause in SQL is translated into **Boolean Indexing** in Pandas. The AI creates a mask (e.g., `df['salary'] > 50000`) that returns only the rows where the condition is True.
- **Projection:** The SELECT part is handled by passing a list of column names `[['name', 'salary']]` to the DataFrame.

Task 5 – Real-Time Application: Algorithm Translation Across Languages

Scenario:

A company maintains algorithms written in different programming languages.

Examples:

- Converting a Python searching algorithm into C
- Translating a Java sorting program into JavaScript

Instructions:

- Use AI to translate a selected algorithm between two programming languages
- Execute and test both versions
- Document translation challenges such as:
 - Syntax differences
 - Library support
 - Memory management

Expected Output:

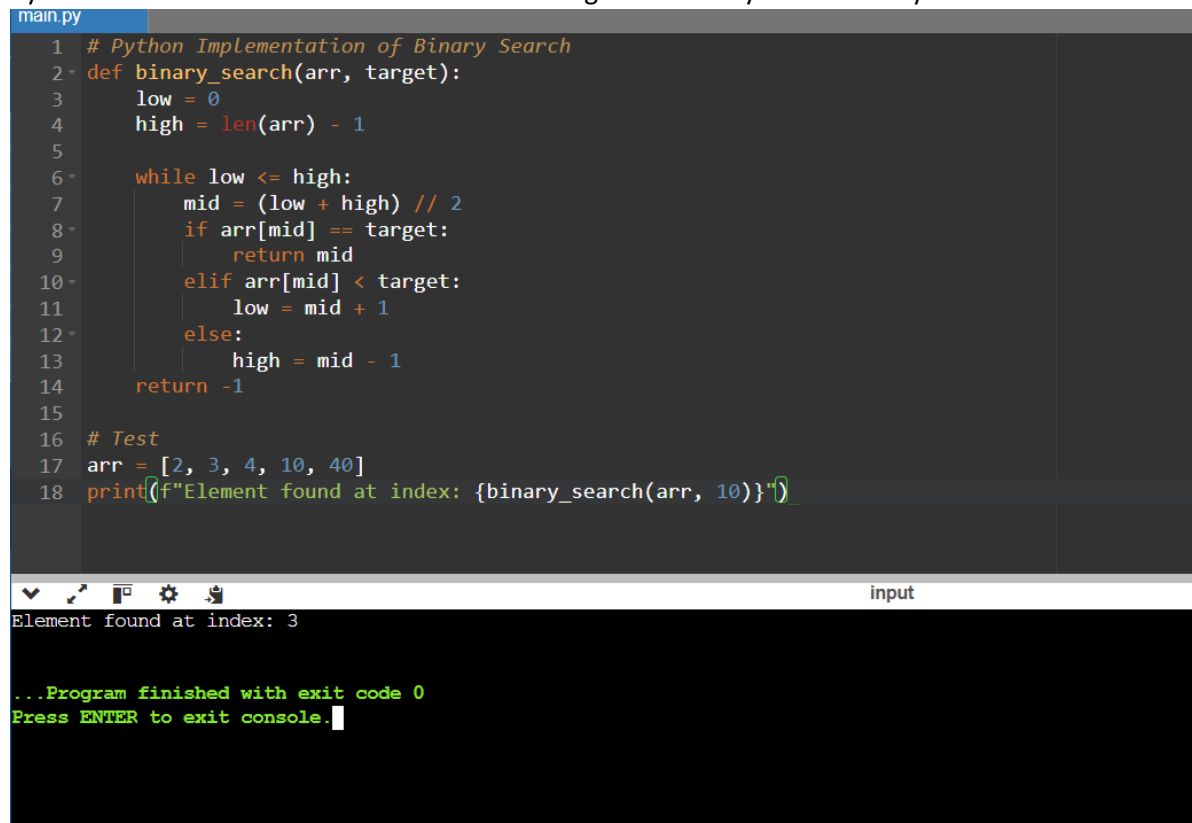
Equivalent, tested code implementations in two different programming languages.

Prompt:

"Translate this Python Binary Search algorithm into C. In the C version, ensure you handle array sizing manually (since C doesn't have a len() function for pointers) and use appropriate integer types. Include a main function that initializes a sorted array and prints the result using printf."

Algorithm (Python)

Python's version is concise and handles list slicing and memory automatically.



```
main.py
1 # Python Implementation of Binary Search
2 def binary_search(arr, target):
3     low = 0
4     high = len(arr) - 1
5
6     while low <= high:
7         mid = (low + high) // 2
8         if arr[mid] == target:
9             return mid
10        elif arr[mid] < target:
11            low = mid + 1
12        else:
13            high = mid - 1
14    return -1
15
16 # Test
17 arr = [2, 3, 4, 10, 40]
18 print(f"Element found at index: {binary_search(arr, 10)}")

input
Element found at index: 3

...Program finished with exit code 0
Press ENTER to exit console.
```

Implementation (C)

The C version requires manual array size calculation and explicit header inclusions.

```
main.c
1  #include <stdio.h>
2  // C Implementation of Binary Search
3  int binarySearch(int arr[], int size, int target) {
4      int low = 0;
5      int high = size - 1;
6
7      while (low <= high) {
8          int mid = low + (high - low) / 2; // Avoids potential overflow
9
10         if (arr[mid] == target)
11             return mid;
12         if (arr[mid] < target)
13             low = mid + 1;
14         else
15             high = mid - 1;
16     }
17     return -1;
18 }
19 int main() {
20     int arr[] = {2, 3, 4, 10, 40};
21     int n = sizeof(arr) / sizeof(arr[0]); // Manual size calculation
22     int target = 10;
23     int result = binarySearch(arr, n, target);
24     if (result != -1)
25         printf("Element found at index: %d\n", result);
26     else
27         printf("Element not found\n");
28     return 0;
29 }
```

input

Element found at index: 3

...Program finished with exit code 0

Explanation:

Algorithm Translation (C vs. Python)

- **Manual vs. Automatic:** This task highlights the biggest challenge: **Abstraction levels**.
- **The "Size" Problem:** In Python, a list "knows" its size (`len(arr)`). In C, an array is just a pointer to a memory location. The AI must add a size parameter to the function because C cannot determine the length of a passed array automatically.
- **Safety:** C is "closer to the metal." The AI uses `low + (high - low) / 2` for the midpoint to prevent **Integer Overflow**, a common bug in low-level languages that Python handles automatically by using arbitrary-precision integers.